

62x
projected speedup

What am I doing?

- Custom AI Chiplet: Performs "Image Quantization" to reduce any photo to 16 optimized colors.
- Unsupervised Learning: Accelerates the K-Means clustering algorithm to group similar pixels autonomously.
- Centroid-Based Math: Uses a custom engine to find the closest "color center" (centroid) for every pixel.

How is it done today?

- CPU: scikit-learn + OpenBLAS — ~9 sec/image, <1% utilization
- GPU: NVIDIA cuML / CUDA — fast but general-purpose, not K-Means specific
- FPGA: AMD Artix-7 pipelined designs — rigid, device-specific
- No dedicated near-memory chiplet for K-Means quantization with HBM3 + UCle

What am I doing differently?

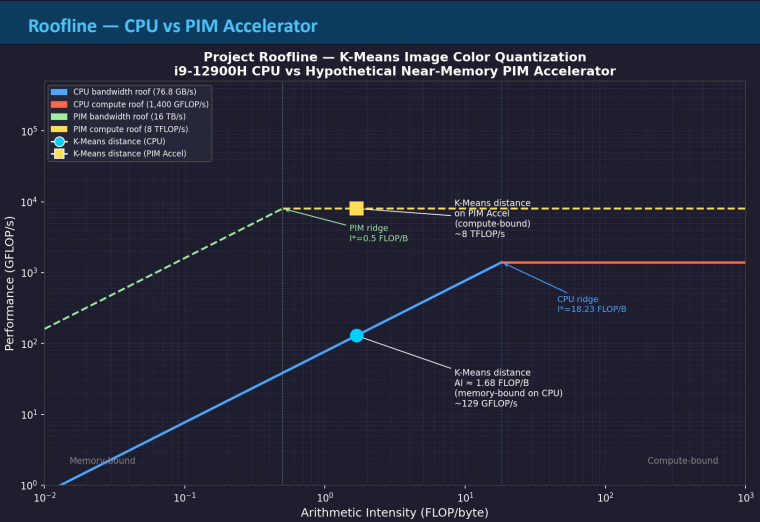
- Memory & Math Side-by-Side: Avoids the "Von Neumann bottleneck" by eliminating data travel time.
- Massive Parallelism: Utilizing HBM3 to feed the Distance Engine, evaluating all 16 centroids simultaneously without memory starvation.
- Shift to Compute-Bound: 16 TB/s bandwidth → ~62x speedup by ensuring the math units never sit idle.

Accomplished so far

- Benchmarked CPU Baseline: Confirmed the 9-second bottleneck where the CPU sits idle waiting for RAM.
- Hardware Logic Verified: Distance engine and AXI4-Lite control interface passing simulation.
- Interconnect: UCle selected as the high-speed host interface to the computer.

What's next?

- Top-level integration: host driver ↔ UCle ↔ AXI4-Lite slave ↔ integer distance core
- System-level functional verification with SystemVerilog TB
- Performance benchmarking
- RTL synthesis & timing reports



System Architecture

K-Means Image Color Quantization – High-Level System Block Diagram

